

GRADUATION PROJECT

Semi-Humanoid, Wheeled-Base General-Purpose Robot

ELECTRICAL SYSTEMS DESIGN PLAN

Power Train — PCB Design — Sensors — Control Architecture

Scope: Electrical & Electronic Systems Only

Mechanical design, ML/software stack, and industrial design are outside this document's scope

Prepared for a one-semester hardware development timeline

Component sourcing referenced against the Egyptian electronics market (Future Electronics, Maamoon Est., Micro Ohm, DIY Electronics Egypt, Dev Boards Market, and equivalents)

1. System Overview & Design Philosophy

1.1 What the robot is, electrically

Physically: a differential-drive wheeled base carrying a torso, two 5–6 DOF arms each ending in a single-motor underactuated gripper hand, and a pan-tilt sensor head. Electrically, it is a distributed embedded system: one high-level compute module for perception/AI/task-switching, and multiple real-time microcontroller nodes — one per major subsystem — communicating over a shared bus, each responsible for closing its own low-level control loops (motor position/current, encoder feedback, local sensors) so that the high-level computer never has to bit-bang real-time motor control.

This separation is the single most important architectural decision in the whole plan, so it's stated up front: the high-level computer plans and perceives; the microcontrollers execute and protect. This keeps the system safe (an MCU can enforce current/torque limits even if the AI software hangs or crashes), keeps the high-level side simpler (it just sends target positions/velocities over a bus and reads back state), and keeps the design modular.

1.2 Three-tier control hierarchy

Tier	Hardware	Responsibility	Update Rate
Tier 1 — High-Level	Raspberry Pi 5 (8GB) — Jetson Orin Nano optional upgrade	Vision/perception, task logic, USB task-loading, HRI (speech/face), path planning, sends high-level commands over CAN/UART bridge	10–30 Hz (task-dependent)
Tier 2 — Subsystem MCUs	3× STM32 (Arms×2 or combined Body MCU, Base MCU) + 1× ESP32 (Head MCU)	Real-time closed-loop motor control, encoder reading, local sensor fusion, safety interlocks, reports state upstream	1– kHz control loop
Tier 3 — Actuator/Sensor level	Motor drivers, encoders, current sensors, IMUs, force sensors	Physical actuation and raw sensing	Analog / driver-internal

Communication backbone: CAN bus (via MCP2515/TJA1050 transceivers, or STM32 built-in bxCAN) is used between Tier 1 and Tier 2 for its robustness, multi-drop wiring (critical in a robot with long cable runs from torso to wheels/arms), built-in error detection, and deterministic behavior under electrical noise from motor switching — which will be significant given the number of PWM-driven motors sharing the chassis. UART/USB is used as a fallback/debug channel and for the head MCU if CAN transceiver count needs to be minimized.

2. Power Train Architecture

2.1 Power budget philosophy

A humanoid-form robot with 10–12+ actuated joints, a compute module, and a full sensor suite has a wide dynamic power range: near-idle when just perceiving, to a high peak when several joints move simultaneously against gravity/load plus the base drives. The power system is sized for worst-case simultaneous peak draw with margin, not average draw — undersizing here is the most common cause of brownouts, MCU resets, and mysterious motor stalls in student robotics projects.

2.2 Battery chemistry decision

Recommendation: LiFePO₄ (LFP), specifically a 4S (12.8V nominal, ~14.6V full) or 6S (19.2V nominal) pack, over standard Li-ion/Li-poly (LiCoO₂/NMC).

- Thermal/chemical safety margin is much higher than LiPo — important for a project handled repeatedly by students in a lab, transported, and possibly demoed publicly.
- Flat discharge curve keeps logic-rail regulators in their efficient input range for most of the discharge cycle, simplifying buck converter selection.
- Longer cycle life (2000–4000+ cycles) tolerates the heavy charge/discharge abuse of a semester of daily testing.
- Widely available in Egypt in the 3S/4S "12V-class" pack sizes used for e-bikes/UPS conversions, with matching BMS boards stocked locally.

2.3 Voltage rail architecture

A single battery voltage is distributed to multiple regulated rails via a custom Power Distribution Board (PDB) — see Section 4. Rails required:

Rail	Voltage	Approx. Current	Powers
Motor Bus (raw)	12.8V (4S LFP) or 19.2V (6S LFP)	Up to 15–20A peak	All motor drivers directly (drivers regulate/PWM down as needed)
5V Logic Rail	5V ±5%	5–8A	MCUs, sensors, servo-class actuators, CAN transceivers, USB hub
3.3V Rail	3.3V	2–3A	MCU cores (where not on-board regulated), IMUs, some sensors
Compute Rail	5V (USB-C, 5V/5A)	Up to 5A burst	Raspberry Pi 5 (needs clean, high-current 5V — the Pi 5 is notably more current-hungry on 5V than older Pi boards)
Servo/Actuator Aux	6–8.4V (if hobby-class servos used anywhere, e.g. head pan-tilt)	2–3A	Pan-tilt servos, camera gimbal

2.4 Battery Management System (BMS)

A dedicated BMS is mandatory, not optional, for this project: it protects against over-discharge, over-charge, over-current, cell imbalance, and short circuit — all real risks with a hand-assembled pack under student supervision. Specify a 4S (or 6S) LFP-rated BMS with balance leads, continuous current rating at least 1.5× your worst-case peak draw, and ideally UART/I²C telemetry output (state of charge, per-cell voltage, temperature) that the Base MCU can read and relay upstream — this lets the robot self-report low battery and trigger a safe-stop/return-to-dock behavior rather than browning out mid-task.

2.5 Main power switch, fusing, and emergency stop

Non-negotiable for a robot with exposed moving joints and a mobile base:

1. A main power disconnect switch (or contactor for the motor bus) accessible without tools, breaking the battery-to-system connection.

2. A mechanical, latching Emergency Stop (E-stop) button wired in series with the motor bus contactor/relay — pressing it must cut motor power in hardware, independent of any MCU or software state.
3. Fusing at the battery output (a single main fuse sized to pack discharge rating) plus per-rail fuses or resettable PTCs on the PDB so a fault on one subsystem (e.g., a shorted arm motor) cannot take down the whole robot.
4. Reverse-polarity and over-voltage protection (MOSFET-based ideal diode or simple Schottky + fuse) at the battery input to the PDB, protecting against miswiring during the many pack swaps a semester of testing involves.

2.6 Charging System

Not addressed elsewhere in the power section — needed for day-to-day operation given the semester's repeated testing cycles.

- Charge port: a dedicated XT60 (or XT90 for higher current) charge connector, separate from the main discharge connector, wired directly to the battery pack's charge leads through the BMS — avoids routing charge current through the main disconnect switch or PDB.
- Balance charging: charge through the BMS's balance leads (JST-XH balance connector, standard on 4S/6S LFP BMS boards per Section 2.4) using an LFP-profile charger — using a LiPo-profile charger on an LFP pack under- or over-charges it, so the charger chemistry setting matters as much as the connector.
- Off-board charging is the simpler default for a lab setting (remove the pack or plug in an external LFP charger) — on-board charging circuitry adds cost and complexity that isn't justified unless the robot needs to self-dock and recharge autonomously, which is outside this project's stated scope.

3. Actuator & Motor Selection

3.1 Selection logic

Joint torque requirements fall roughly into three tiers on a 5–6 DOF humanoid arm: shoulder (pitch/roll — highest torque, lifts the whole arm against gravity plus payload), elbow (moderate torque), and wrist/gripper (lowest torque, highest speed/precision need). Rather than using one motor class everywhere, this plan tiers the actuator choice to match it. This is both more torque-appropriate and, importantly, cheaper, since you are not over-specifying expensive motors for low-torque joints.

3.2 Arm joints — shoulder & elbow (high torque tier)

Recommendation: NEMA 17 or NEMA 23 stepper motors (closed-loop, i.e. with an integrated or add-on magnetic/optical encoder) driven through a timing belt or planetary gear reduction (mechanical scope, but drives your encoder resolution choice electrically).

- Closed-loop steppers give you position feedback and stall detection — critical for a robot whose arms will be commanded by an unpredictable, swappable software stack, so you can detect a jammed joint and cut power before something breaks.
- NEMA 17/23 steppers and matching drivers (A4988, DRV8825, TMC2209/TMC5160, or higher-current TB6600-class for NEMA23) are all readily available in the Egyptian market (Future Electronics, Maamoon Est.), unlike imported smart-servo actuators.
- TMC-family drivers (TMC2209, TMC5160) specifically are worth the small price premium over A4988: silent operation (important for a demo-facing project), integrated current sensing, and stall detection (StallGuard) that can be read by the MCU as a software limit switch / collision-detection signal — a genuinely useful safety feature for an arm operating near people. TMC2209 is the baseline driver across all stepper joints; TMC5160 is an optional upgrade reserved for the higher-torque NEMA23 shoulder joints if TMC2209's current rating proves insufficient once the mechanical payload is finalized (see BOM, Section 9.2).

3.3 Wrist & forearm rotation (medium torque tier)

Smaller NEMA 17 steppers (lower current draw, e.g., 1.3–1.7A/phase) or small brushed DC gear-motors (encoder-equipped) with lighter TMC2209 or DRV8825 drivers.

3.4 Hand / gripper (single motor, underactuated)

Given your choice of one motor per hand with underactuated fingers: a single geared DC motor (6–12V, moderate stall torque, e.g., a compact 12V planetary gear-motor) driving a tendon or linkage that closes all fingers together, with a current-sensing driver (or an inline current-sense resistor + amplifier feeding an ADC) used as the grip-force proxy — since there's no dedicated force sensor per finger, motor current at stall is your practical signal for "object gripped, stop closing further, and modulate holding force." This is a well-established technique for underactuated grippers and gives you soft-object handling without a full tactile-sensor array.

- Driver: a single-channel DC driver such as BTS7960 (high current headroom, thermal protection) or a small DRV8871-class IC on your custom hand PCB (Section 4.4).
- Add a limit switch or magnetic (Hall) end-stop at the fully-open position as a hard reference the firmware can home to on startup — cheap insurance against encoder drift over long uptimes.

3.5 Wheeled base drive

Recommendation: differential drive — two driven wheels (one per side) plus one or two passive caster wheels, rather than mecanum/omni or 4WD skid-steer.

- Mecanum wheels add holonomic motion (strafing) that is rarely useful for a general-purpose indoor robot whose primary need is "navigate to a location and stand still while the arms work" — the added omnidirectional capability doesn't pay for its cost: 4 driven motors instead of 2, 4 motor drivers instead of 2, more complex inverse kinematics in firmware, and mecanum wheels/hardware are more expensive and less commonly stocked locally than plain wheels.
- 4WD skid-steer adds redundant driven wheels for a task that differential drive already handles (indoor floors, moderate loads) — it mainly adds value outdoors/rough terrain, which is not your stated use case.
- Differential drive is the simplest to control (single PID per side, standard unicycle kinematics), cheapest (2 motors, 2 drivers, 2 encoders), lightest on the power budget, and has the most abundant, cheapest, best-documented local parts (DC gear-motors with encoders in the 12V – 24V class are a commodity item at every Egyptian robotics retailer).

Motor spec target: 12V or 24V DC gear-motors, encoder-equipped, sized for your total robot mass, a 15–25 kg robot comfortably needs two ~200–300 RPM (post-gearbox) 12V–24V motors with reasonable stall torque; confirm against final mass and wheel diameter using a standard $\text{torque} = (\text{mass} \times g \times \text{wheel_radius} \times \text{safety_factor}) / 4$ calculation once the mechanical BOM is closed). Driver: BTS7960 (cheap, high current, thermally protected, very common in the Egyptian market) or a dual-channel motor driver board of similar class, one per wheel or a dual-channel board for both.

3.6 Head pan-tilt

Given the full sensor suite requirement (camera + mic array), the head needs 2 DOF (pan + tilt) carrying modest load (camera, maybe a small display for a "face"). This is the one place hobby-class analog or digital RC servos (e.g., MG996R-class metal-gear servos, widely available in Egypt) are the pragmatic choice over steppers — low load, need for compact integrated gearing, and simple PWM control that doesn't warrant a full closed-loop stepper setup.

3.7 Actuator summary table

Joint / Axis	Actuator Class	Qty	Driver	Feedback
Shoulder pitch/roll (×2 arms)	NEMA17/23 closed-loop stepper (or DC gear-motor)	4	TMC2209 (baseline) / TMC5160 (optional upgrade)	Integrated encoder + StallGuard
Elbow (×2 arms)	NEMA17 closed-loop stepper (or DC gear-motor)	2	TMC2209	Integrated encoder
Wrist rotate/flex (×2 arms)	Small NEMA17 stepper or DC gear-motor	2–4	TMC2209 / DRV8825	Encoder
Gripper (×2 hands)	DC gear-motor, single-motor underactuated	2	BTS7960 / DRV8871	Current sense + open limit switch
Base drive wheels	12/24V DC gear-motor	2	BTS7960 (×2 or 1 dual-ch)	Quadrature encoder
Head pan/tilt	Metal-gear RC servo	2	Direct PWM (buffered)	Internal potentiometer

3.8 Stepper vs. DC Gear-Motor — Detailed Comparison (Arm Joints)

Section 3.2 already selects closed-loop steppers for the shoulder and elbow joints. This subsection makes that trade-off explicit, in the same comparative format used for the PCB architecture decision in Section 12.

Factor	Closed-loop Stepper (NEMA17/23)	Brushed DC Gear-Motor (encoder-equipped)
Position accuracy	Excellent — open-loop step accuracy backed by encoder for closed-loop correction; no drift	Good, but fully dependent on encoder + PID tuning; more prone to overshoot
Holding torque at rest	Very high, holds position with power applied and minimal control effort	Requires active PID holding current; drifts under load if the control loop stalls
Torque at low speed	Excellent — steppers are strongest at low RPM, matching arm joint speed profiles	Weaker at low RPM without a heavy gearbox; needs a higher reduction ratio
Stall / collision detection	Built-in via TMC StallGuard — no extra hardware	Needs current-sensing + threshold logic in firmware (extra ADC channel, more tuning)
Smoothness / audible noise	Silent with TMC2209/5160 (microstepping); can “cog” with cheaper drivers	Naturally smooth, no microstepping needed
Driver cost & availability (Egypt)	TMC2209 ~180–320 EGP, TMC5160 ~500–900 EGP — common	BTS7960 ~120–220 EGP — cheaper, very common
Control complexity	Step/dir + UART/SPI config; more setup but simpler safety logic	PID tuning per joint required; more firmware effort for equivalent safety behavior
Efficiency	Lower — draws current even when holding (unless standstill current-reduction mode is used)	Higher — only draws current when doing work
Backlash / gearing	Depends on external reduction (belt / planetary) — mechanical, not motor-inherent	Same — depends on gearbox choice

Recommendation (unchanged from Section 3.2, now with the comparison on record): keep closed-loop steppers for shoulder/elbow. For a robot with unpredictable, swappable task software, StallGuard-based collision detection without extra sensing hardware is the deciding factor — DC motors would need current-threshold stall detection built and tuned per joint in firmware, adding real risk given the one-semester timeline. DC gear-motors remain the right call where already used (base drive, gripper) because continuous holding torque isn't the priority there.

3.9 Telescopic Lift Column Actuation

The mechanical design specifies a multi-stage electric telescopic column (≈ 40 cm to ≈ 150 cm stroke, ~ 110 cm travel) raising the entire upper body, driven by a lead-screw or ball-screw actuator. This subsystem is not currently represented anywhere in the electrical plan and needs its own driver, power allocation, and MCU ownership.

- Actuator: single DC gear-motor or NEMA23 stepper driving the lead screw/ball screw, matched to whichever the mechanical team finalizes — if a stepper, it can reuse the same TMC5160/TMC2209 driver family already stocked for the shoulder joints, keeping the BOM simpler than introducing a new driver class for one axis.
- Position feedback: absolute or incremental encoder on the motor shaft or a moving stage, as specified in the mechanical plan — required for the high-level software to know current column height before planning a reach.
- Limit switches: mechanical or Hall-effect switches at both travel extremes, wired as a hardware cutoff (same pattern as the gripper's open-position limit switch in Section 3.4) — independent of software, so a runaway command can't drive the column into its own end stops.
- Owning MCU: logically the Base MCU (Section 4.3), since it already owns power-adjacent monitoring and is physically closer to the torso base than the Head MCU — add one motor driver channel, one encoder input, and two limit-switch GPIOs to that board's scope.
- Power: draws from the Motor Bus (raw) rail (Section 2.3) like the other joint actuators; size its fuse/PTC on the PDB for its expected peak current once the mechanical team confirms motor selection.
- Note the lead-screw's self-locking property (mechanical plan, Section 4.2) means the column holds position without continuous motor current — favorable for the power budget compared to the arm joints.

4. Custom PCB Architecture

Four custom boards are recommended, aligned with the three-tier hierarchy plus power. This modularity means a fault or revision in one board doesn't require re-spinning the whole system — valuable when working to a one-semester deadline where you cannot afford a full re-fab if one subsystem has a bug.

4.1 Board 1 — Power Distribution Board (PDB)

The electrical heart of the robot. Takes raw battery input and produces every regulated rail in Section 2.3.

- Input: battery bus through the main disconnect switch, reverse-polarity protection MOSFET, main fuse, and E-stop relay/contactors.
- Output stages: high-current buck converter modules (or your own synchronous buck design) for 5V and 3.3V rails, a dedicated clean 5V/5A-capable buck for the Raspberry Pi compute rail, and a switched/fused breakout for the raw motor bus to each driver board.
- Per-rail: polyfused or blade-fuse protection, power-good LED indicators, and test points — test points are cheap to add and save enormous debugging time.
- Include a current sense (e.g., INA219/INA226 I2C power monitor IC) on the main battery line so the Base MCU can log/report total system power draw and battery percentage in real time.
- Physically centralize this board near the battery bay in the base, with heavy-gauge wiring (properly sized AWG for your peak current, not just "whatever's on hand") running up to torso/arm/head boards.

4.2 Board 2 — Body MCU Board (Arms + Torso)

Hosts an STM32 (recommend STM32F4-series for its motor-control timers, multiple ADCs, and CAN peripheral — e.g., STM32F407/F411 class, available locally or via common dev boards like "Black Pill"). Responsibilities: closed-loop control of all arm joint motors (both arms, i.e. up to 10–12 stepper/DC channels total — consider

one Body MCU per arm if channel count and firmware complexity get high, which is a very reasonable split given your "one MCU per subsystem" preference).

- On-board or breakout headers for TMC2209/5160 driver modules (UART or SPI configured for silent mode and StallGuard readout).
- Encoder input channels (quadrature or SPI/I2C absolute encoders depending on final choice).
- Current-sense inputs for the gripper motors (ADC channels, or read directly from driver current-sense pins).
- CAN transceiver (e.g., TJA1050/TJA1051) tied to the STM32's bxCAN peripheral, connecting this board to the shared bus.
- Local decoupling and a clean 3.3V/5V regulator stage fed from the PDB's motor-adjacent rail, with its own fusing.

4.3 Board 3 — Base MCU Board (Wheels + Navigation Sensors)

Hosts an STM32 (F1 or F4 class is sufficient here — lower channel count than the arms) responsible for: differential-drive wheel motor control (2× BTS7960 driver interface, PWM + direction + current sense), quadrature encoder reading for odometry, and hosting/aggregating base-level sensors: IMU (for base orientation/tilt), ultrasonic or ToF distance sensors (basic obstacle avoidance), and the BMS telemetry link (Section 2.4).

- This board is the natural place for the main power monitor (INA226) and E-stop status line, since it's physically closest to the PDB and battery bay.
- CAN transceiver for the shared bus, same as Board 2.
- This MCU is a good candidate to also own the "system state machine" (idle / moving / arm-active / fault / low-battery) since it has direct visibility into power state — though the Raspberry Pi ultimately arbitrates high-level task logic.

4.4 Board 4 — Head MCU Board (Sensors + HRI)

An ESP32 (rather than STM32) is the better fit here specifically because the head's job — camera framing/interfacing, audio, Wi-Fi/BLE for possible auxiliary comms, driving small pan-tilt servos — benefits from ESP32's built-in Wi-Fi/BLE and ample general-purpose I2C/I2S peripherals for a mic array, more than it needs STM32-class real-time motor-control timers (pan-tilt is just two low-torque RC servos).

- PWM outputs (buffered, level-shifted if needed) for the two pan/tilt RC servos.
- I2S or analog interface for a microphone (or small mic array, if you want basic sound-source direction — 2–4 MEMS mics is a reasonable "advanced but achievable" target).
- Camera: if using a USB (or CSI ribbon) camera hangs off the Raspberry Pi directly, this board doesn't need to carry video — keep video off the low-bandwidth CAN bus entirely and let the Pi own the camera over USB/CSI directly. The Pi 5 has two 4-lane MIPI CSI connectors, so a CSI camera is actually a strong option here (lower latency, no USB bandwidth contention) if you want a dedicated head-mounted camera separate from a body camera. For "general-purpose robot vision" a camera direct to the Pi (Section 5) is simpler and higher quality than routing video through the Head MCU.
- CAN transceiver (or, if channel/pin-constrained, a UART bridge to the Body MCU which relays over CAN) for sending sensor data upstream and receiving pan/tilt commands.

4.5 PCB design & fabrication notes

- Design in KiCad (free, strong community support, widely taught) unless your institution provides Altium licenses.

- Route motor power traces and driver ICs away from MCU crystal/analog sections; keep a solid ground plane and star-ground the high-current return paths back to the PDB to avoid ground-loop noise corrupting encoder/ADC readings — this is the #1 source of "my encoder readings are noisy" bug reports in student robot projects.
- Add TVS diodes / flyback protection on every motor driver output and on CAN bus lines entering each board — motors and long cable runs generate transients that will kill an unprotected MCU pin.
- Break out SWD (STM32) / JTAG-or-USB (ESP32) programming headers on every board, accessible after the board is mounted in the chassis — easy to forget until a board already installed behind wiring needs a firmware reflash.

5. High-Level Compute & the USB / Reprogrammable Task Model

Your requirement — general-purpose usage, with tasks changed via USB or by reflashing high-level software — is best met by keeping all task-specific logic on the Raspberry Pi and keeping the low-level MCU firmware task-agnostic (it only ever exposes generic primitives: "move joint X to angle/velocity Y," "report encoder/current/IMU state," "close gripper to force Z"). This way, changing the robot's task is purely a high-level software change (new USB-loaded program, new model, new behavior script) and never requires reflashing or modifying the real-time firmware — exactly the electrical/firmware boundary that makes your USB-swappable-task requirement clean to implement.

5.1 Compute module recommendation

Recommendation: Raspberry Pi 5 (8GB). At current Egyptian retail pricing the Jetson Orin Nano runs approximately 40,000 EGP — roughly half to all of the entire electrical budget by itself — which is disproportionate for a general-purpose task-agnostic robot where the low-level MCUs (Section 1.2) already carry the real-time control burden. The Pi 5 (8GB) is confirmed available in Egypt around 13,500 EGP (e.g. Micro Ohm Electronics), a difference of roughly 26,000+ EGP that is far better spent on TMC5160 drivers, better encoders, or budget contingency.

- USB 3.0 ports: used both for your task-swap USB requirement (plug in a USB drive/dongle with a new task package) and for the camera(s).
- GPIO/UART/I2C 40-pin header: use one UART (or the CAN-over-SPI route via an MCP2515 module, since the Pi has no native CAN peripheral) to bridge into the Tier 2 CAN bus.
- Compute: 2.4GHz quad-core Cortex-A76, no dedicated GPU/tensor cores. This is real CPU-class performance — fine for control logic, sensor fusion, task-switching, and lighter CPU-based vision (classical CV, lightweight/quantized detection models) at usable but not flashy frame rates. It is not GPU-class deep learning inference.
- Upgrade path if heavier vision AI proves necessary once tasks are scoped: (a) add a USB AI accelerator (e.g. Coral Edge TPU or Hailo-8 USB) to the Pi 5 for a few thousand EGP rather than tens of thousands — check current Egyptian availability before committing; or (b) swap in a Jetson Orin Nano later, since the electrical/firmware boundary in this plan (Section 5) is compute-module-agnostic — changing Tier 1 hardware does not require touching Tier 2/3 firmware or wiring, only the CAN/UART bridge code on the new module. Document this as your justified upgrade path if asked in your defense.

5.2 Raspberry Pi power integrity

Re-stating Section 2.3's warning because it's the most common integration failure: give the Pi 5 a dedicated, well-regulated 5V/5A (USB-C PD-capable) supply from the PDB, with bulk capacitance and a ferrite bead at the input. The Pi 5 draws meaningfully more current than earlier Pi boards, especially with USB peripherals (camera, USB task-drive, any accelerator) attached, and its firmware actively monitors input voltage — under-

voltage causes visible throttling or a refusal to fully boot rather than a silent failure, which is at least easier to diagnose than the Jetson's harder brownout/reboot behavior, but still needs real design margin on this rail. Brownout resets on the compute module mid-task are disproportionately disruptive since it owns all task logic.

6. Sensor Suite

Matching your "full sensor suite" requirement, organized by subsystem and mapped to which MCU owns them:

Sensor	Purpose	Interface	Owning Board	Egyptian Availability
USB or CSI camera (or stereo pair)	Vision/perception, object recognition for general tasks	USB 3.0 or CSI ribbon	Raspberry Pi (direct)	Common (Logitech-class USB; official Pi Camera Module for CSI)
IMU (6–9 DOF, e.g., MPU6050/MPU9250/BN0055)	Base tilt/orientation, fall/tip detection, arm pose sanity-check	I2C	Base MCU (+ optional 2nd on torso)	Very common locally
Quadrature encoders (wheels + DC joints)	Odometry, position feedback	Digital (2-ch)	Base MCU / Body MCU	Common (magnetic AS5600 or optical)
Stepper closed-loop encoders	Joint position feedback, stall detection	Integrated w/ driver (TMC UART/SPI)	Body MCU	Available via TMC driver modules
Current sense (per motor + main bus)	Grip-force proxy, stall/overcurrent protection, power monitoring	ADC / INA219/INA226 (I2C)	All driver-bearing boards	Very common locally
Ultrasonic (HC-SR04) or ToF (VL53L0X/VL53L1X)	Base obstacle avoidance / proximity	Digital trigger/echo or I2C	Base MCU	Very common locally
MEMS microphone(s) / mic array	Audio input for HRI, optional sound-source localization	I2S / analog	Head MCU	Common (individual mics); arrays may need import
Limit / Hall-effect switches	Homing reference (gripper open, joint zero position)	Digital GPIO	Body MCU	Very common locally
Battery telemetry (via BMS)	State of charge, per-cell voltage, temperature	UART / I2C (BMS-dependent)	Base MCU	Common on 4S/6S LFP BMS boards
Temperature sensors (driver/motor heatsinks)	Thermal protection for high-duty drivers	Analog (NTC) or I2C	Body MCU / PDB	Very common locally

6.1 Force/Tactile Sensing — Should It Be Added?

Current baseline: the gripper already gets a grip-force proxy for free via motor current-sensing (Section 3.4) — no dedicated sensor is required for basic “object gripped, stop closing” behavior.

What a dedicated force sensor adds: actual contact-force magnitude rather than a motor-side proxy, awareness of where on the finger/palm contact occurs, and finer force modulation for delicate objects.

Compatibility with the current architecture — yes, it fits cleanly:

- FSR (Force-Sensitive Resistor) strips or pads at the fingertips: pure analog, read via a resistor divider into any free ADC channel — the Body MCU (STM32F4) already has ADC channels free after current-sense and encoder inputs, so this needs no new board or bus node, just added analog wiring routed alongside the tendon cables through the forearm.
- I2C-based capacitive/piezoresistive tactile sensors: would share the I2C bus already used by the IMUs — feasible, but check Egyptian availability, since these tend to be import-only unlike FSRs.
- Load cell + HX711 amp module at the wrist: easy UART/digital interface, cheap and common locally, but gives only total wrist-reaction force, not per-finger contact — less useful for grip modulation than fingertip FSRs.

Recommendation: add 1–2 low-cost FSRs per fingertip (thumb + one opposing finger is often enough for an underactuated hand) as an optional enhancement, not a required redesign — a small BOM/wiring addition using ADC channels already free on the Body MCU. A full per-finger tactile array is not recommended: cost, import dependency, and integration time aren't justified given the gripper is already single-motor underactuated, with limited independent finger control to act on that data anyway.

7. Wiring, Connectors & Cable Management

Often under-planned in student projects and a major source of last-minute failures — worth explicit attention given the number of joints and the semester timeline.

- Standardize connector families by function: JST-XH or Molex KK for low-current sensor/signal lines, XT30/XT60-class connectors for motor power (rated appropriately per driver's peak current), and a locking connector (e.g., GX12/GX16 aviation connectors) at every joint that physically rotates, so cables can be disconnected for maintenance without cutting/re-terminating.
- Route torso-to-arm and torso-to-head cable bundles through hinge/rotation points with enough slack and strain relief (cable chains or braided sleeving) to survive repeated joint articulation without fatigue-breaking conductors — a mechanical concern with direct electrical consequences (intermittent faults are far harder to debug than a motor that simply doesn't work).
- Keep CAN bus wiring as a proper twisted pair with correct 120Ω termination resistors at each physical end of the bus (typically one at the Base MCU end, one at the far end — e.g., Head MCU or last Arm MCU), not at every node.
- Separate power and signal wiring runs physically (different loom/sleeve) wherever possible, and cross power/signal cables at 90° if they must cross, to minimize inductive coupling noise onto sensor lines.
- Route the Raspberry Pi 5's Wi-Fi/BLE antenna (and the Head MCU's ESP32 antenna) away from the motor bus and CAN wiring, and away from driver ICs — PWM switching noise from the steppers/DC drivers can degrade Wi-Fi range and cause intermittent auxiliary comms drops.

8. Safety Systems Summary

Consolidating safety features referenced throughout — this list is worth presenting as its own section/slide in your project defense, since examiners in Egypt (and generally) weight safety design heavily for anything with moving parts and batteries.

1. Hardware E-stop, latching, cuts motor bus power independent of software (Section 2.5).
2. Main and per-rail fusing/PTCs (Section 2.5, 4.1).
3. Reverse-polarity and over-voltage protection at battery input (Section 2.5).
4. BMS with over-charge/discharge/current/temperature protection on the pack itself (Section 2.4).

5. Per-joint stall detection (StallGuard on steppers, or current-threshold on DC motors) that the owning MCU uses to cut that joint's power on jam/collision, without needing the high-level computer to intervene (Section 3.2).
6. Software-independent watchdog timers on every MCU: if a Tier 2 MCU stops receiving valid CAN commands from the Raspberry Pi for a defined timeout (e.g., 200–500ms), it must independently ramp motors to a safe stop — protecting against a high-level software hang or crash, which is a very real risk given the swappable/experimental nature of your task software.
7. Current monitoring at both the main bus (INA226 on PDB) and per-subsystem, logged and available to trigger graduated responses (warn → throttle → stop) rather than a single hard cutoff, so a temporary current spike doesn't unnecessarily halt the whole robot.
8. Physical limit/Hall switches at joint extremes as a hardware backup to software-enforced joint limits.
 - System-state indicator LEDs (power on, fault, E-stop tripped, low battery) visible from outside the chassis — cheap to add on the PDB/Base MCU, and gives visible confirmation of system state during bring-up, debugging, and public demos without needing a laptop attached.

9. Bill of Materials (Indicative, Egyptian Market)

Prices are indicative single-unit retail figures gathered from Egyptian electronics retailers; re-quote before purchasing, and note that per-unit pricing typically drops if these retailers offer education/bulk discounts — worth asking directly given this is a university project.

9.1 Power Train

Component	Qty	Unit Price (EGP)	Subtotal (EGP)	Egyptian Source
4S LiFePO4 battery pack (12.8V, capacity per final power budget, e.g. 10–20Ah)	1	2,500–4,500	2,500–4,500	Battery/e-bike suppliers, Maamoon Est.
4S LFP BMS board (with balance leads, UART/I2C telemetry if available)	1	300–700	300–700	Maamoon Est., battery suppliers
Main power switch / contactor	1	150–350	150–350	Future Electronics, local electrical suppliers
E-stop button (latching, panel-mount)	1	150–300	150–300	Future Electronics, Maamoon Est.
Buck converter modules (5V/3.3V high-current, e.g. LM2596/XL4015-class or custom)	3–4	60–150	240–600	Maamoon Est., Future Electronics
INA219/INA226 current/power monitor modules	3–4	80–150	320–600	Future Electronics, Maamoon Est.
Fuses/PTC resettable fuses (assorted) + fuse holders	1 set	150–300	150–300	Maamoon Est., local electronics shops

9.2 Actuators & Drivers

Component	Qty	Unit Price (EGP)	Subtotal (EGP)	Egyptian Source
NEMA17 closed-loop stepper motor (shoulder/elbow, higher torque variant)	4–6	350–700	1,400–4,200	Future Electronics, Maamoon Est., Lancer3D
NEMA17 stepper motor (wrist, standard torque)	2–4	250–450	500–1,800	Future Electronics, Maamoon Est.
TMC2209 stepper driver module	6–8	180–320	1,080–2,560	Future Electronics, Maamoon Est., Amazon.eg
TMC5160 driver module (for higher-torque shoulder joints, optional upgrade)	2–4	500–900	1,000–3,600	Amazon.eg / import via local resellers
12–24V DC gear-motor w/ encoder (base wheel drive)	2	350–650	700–1,300	Future Electronics, Maamoon Est.
BTS7960 high-current motor driver module	4–6	120–220	480–1,320	Maamoon Est., Future Electronics
DC gear-motor (gripper actuation, compact 12V)	2	150–300	300–600	Maamoon Est., Future Electronics
Metal-gear RC servo (head pan/tilt, MG996R-class)	2	150–250	300–500	Maamoon Est., Future Electronics, Amazon.eg
Magnetic encoder modules (AS5600-class, for DC motor joints/gripper)	4–6	120–220	480–1,320	Future Electronics, Amazon.eg
Telescopic column actuator: NEMA23 stepper (or 24V DC gear-motor per final mechanical spec) + dedicated TMC5160/TMC2209-family driver + magnetic encoder module (Section 3.9)	1	900–1,800	900–1,800	Future Electronics, Maamoon Est.

9.3 Compute & Control

Component	Qty	Unit Price (EGP)	Subtotal (EGP)	Egyptian Source
Raspberry Pi 5 (8GB) + official active cooler + 27W USB-C PD supply	1	13,500–15,500	13,500–15,500	Micro Ohm Electronics, Future Electronics, Dev Boards Market
(Optional upgrade) NVIDIA Jetson Orin Nano 8GB Dev Kit — swap-in if heavy on-device inference proves necessary	0–1	~40,000	0 or 40,000	Future Electronics Egypt, Dev Boards Market
(Optional, cheaper AI upgrade) Coral USB Edge TPU or Hailo-8 USB accelerator	0–1	3,000–6,000	0–6,000	Import via local resellers — confirm stock
STM32F4-class dev board (Body/Arm MCU, e.g. Black Pill F411)	2 (one per arm) or 1 combined	250–450	500–900	Future Electronics, Maamoon Est., Amazon.eg
STM32F1/F4-class dev board (Base MCU)	1	200–400	200–400	Future Electronics, Maamoon Est.

Component	Qty	Unit Price (EGP)	Subtotal (EGP)	Egyptian Source
ESP32 dev board (Head MCU)	1	180–300	180–300	Very common — Future Electronics, Maamoon Est.
MCP2515 + TJA1050 CAN transceiver modules	4–5	80–150	320–750	Future Electronics, Maamoon Est., Amazon.eg
USB camera (USB3, reasonable resolution/FPS)	1–2	600–1,500	600–3,000	Amazon.eg, Future Electronics
MEMS microphone module(s)	1–4	50–150	50–600	Future Electronics, Maamoon Est.

9.4 Sensors & Misc.

Component	Qty	Unit Price (EGP)	Subtotal (EGP)	Egyptian Source
IMU module (MPU6050/MPU9250/BNO055)	2	150–450	300–900	Future Electronics, Maamoon Est.
ToF distance sensor (VL53L0X/L1X) or HC-SR04 ultrasonic	2–4	80–200	160–800	Future Electronics, Maamoon Est.
Limit switches / Hall-effect end-stops	8–12	15–40	120–480	Maamoon Est., local electronics shops
NTC thermistors (driver/motor thermal monitoring)	6–10	10–25	60–250	Local electronics shops
Connectors (XT30/XT60, JST-XH, GX12/16 aviation), assorted	1 set	500–1,000	500–1,000	Maamoon Est., local electronics shops
Wiring, sleeving, heatshrink, cable chain	1 set	400–800	400–800	Local electronics/hardware shops
(Optional) Fingertip FSR force sensors, 1–2 per hand (Section 6.1)	0–4	40–90	0 or 80–360	Future Electronics, Maamoon Est., Amazon.eg

9.5 Custom PCB Fabrication

Component	Qty	Unit Price (EGP)	Subtotal (EGP)	Egyptian Source
PCB fabrication (4 unique boards, ~3 copies each for spares/rework, incl. shipping to Egypt)	12 boards (paneled)	170–420	2,000–5,000	JLPCB/PCBWay (import) or local Egyptian PCB fab houses
SMD/THT components for PCB assembly (passives, connectors, regulators)	1 set	—	1,000	Future Electronics, Maamoon Est., LCSC/Digikey via reseller

Component	Qty	Unit Price (EGP)	Subtotal (EGP)	Egyptian Source
PCB assembly tools/consumables (solder, flux, if not already owned)	1 set	—	300–800	Local electronics shops

9.6 Estimated Total

Category	Low Estimate (EGP)	High Estimate (EGP)
Power Train	3,810	7,350
Actuators & Drivers	6,140	18,500
Compute & Control	15,430	20,950
Sensors & Misc.	1,940	4,230
Custom PCB Fabrication	3,300	6,800
Subtotal	30,620	57,830
Contingency (15–20%)	4,590–6,120	8,670–11,570
TOTAL (approx.)	~35,200–36,700	~66,500–69,400

10. Suggested Semester Timeline (Electrical Track)

Phase	Weeks	Key Deliverables
1. Requirements freeze & power budget	1–2	Final DOF count, mass estimate from mechanical team, worst-case current draw calculation, battery/BMS sizing locked
2. Component sourcing & schematic capture	2–4	All BOM items quoted/ordered, KiCad schematics for all 4 boards reviewed
3. PCB layout & fab submission	4–6	Gerbers submitted for all 4 boards (submit early — fab + shipping to Egypt takes 1–2+ weeks)
4. Bring-up & bench testing (per-board, no mechanics yet)	6–9	Each board power-tested, MCU firmware bring-up, motor spin-up on bench, CAN bus communication verified between all nodes
5. Sensor integration & closed-loop tuning	9–11	Encoder feedback verified, PID/stepper tuning per joint, current-sense grip detection working, safety interlocks (E-stop, watchdogs) tested
6. Mechanical integration	11–13	Boards + wiring installed in chassis, cable routing finalized, full-system smoke test
7. System integration with high-level software	13–15	Raspberry Pi ↔ CAN bus command/telemetry loop working, USB task-swap mechanism demonstrated
8. Final testing, safety validation, documentation	15–16	Full safety checklist (Section 8) verified, demo run-throughs, final report/BOM finalized

11. Summary of Key Recommendations

1. Three-tier hierarchy: Raspberry Pi 5 (planning/perception) → STM32/ESP32 subsystem MCUs (real-time control) → drivers/sensors, linked by CAN bus — with a documented, low-friction upgrade path to a Jetson Orin Nano if heavier on-device AI inference becomes necessary later.
2. LiFePO4 battery + dedicated BMS + hardware E-stop + fused, protected PDB — safety-first power architecture appropriate for student handling and public demos.
3. Tiered actuator selection: closed-loop steppers (TMC-driven) or encoded DC gear-motors for high/medium-torque arm joints, simple DC + current-sense for the single-motor gripper, RC servos only for the low-load head pan-tilt, DC gear-motors for differential-drive base — all chosen from classes well-stocked in the Egyptian market, deliberately avoiding import-dependent smart-servo ecosystems given the one-semester timeline.
4. Four custom PCBs (Power Distribution, Body/Arm MCU ×1–2, Base MCU, Head MCU) rather than one monolithic board — isolates faults and enables parallel development.
5. Task-agnostic firmware boundary: low-level MCUs expose only generic motion/sensing primitives; all task-specific behavior lives in Raspberry Pi-side software, cleanly supporting your USB-swappable / reprogrammable general-purpose use case without ever touching real-time firmware.
6. Full sensor suite mapped explicitly to owning boards (Section 6), with grip-force handled via current-sensing rather than a dedicated tactile array, consistent with the single-motor gripper choice.
7. Newly closed gaps from the review pass: telescopic lift column actuation now has an owning MCU, driver, and encoder (Section 3.9); a charging path is defined (Section 2.6); and fingertip force sensing is scoped as an optional, low-cost add-on rather than left as an open question (Section 6.1).
8. Layered safety system (Section 8) combining hardware E-stop, fusing, BMS protection, per-joint stall detection, and software watchdogs — present this explicitly in your defense.

12. Per-Motor Dedicated PCB vs. Consolidated Multi-Axis Driver Board

12.1 The two architectures being compared

Option A — One small PCB per motor ("smart actuator" style): each joint gets its own compact board carrying the driver IC (e.g., TMC5160), local decoupling, and a connector for power + bus (CAN or SPI-over-cable). The board mounts physically at or near the joint itself, and only power + a shared bus cable run back to the Body MCU.

Option B — Consolidated driver board per subsystem (what the current plan specifies): one PCB (e.g., "Left Arm MCU" board) hosts the STM32, multiple driver ICs/modules, and all the SPI/UART/encoder wiring for every joint in that arm, centrally located in the torso or upper arm, with individual motor power/signal cables running out to each joint.

12.2 Comparison

Factor	A — Per-Motor PCB	B — Consolidated Board
Wiring complexity	Lower per-joint (short local connections), but needs a bus cable (CAN/power) routed to every joint through the arm's rotation points	Higher — every motor's power + feedback wires run individually back to one central board, more conductors crossing each joint
Fault isolation	Excellent — a failed board only takes down one joint	Weaker — a fault on the shared board (bad regulator, damaged trace) can affect multiple joints at once
Serviceability/repair	Swap one small board without disturbing other joints	Replacing the board means re-terminating wiring for every joint on that subsystem

Design & fab effort	One design, replicated 8-10+ times (design once, fab many) — actually less total design work despite more physical boards	Fewer unique designs (per plan: 4 boards total), but each is more complex to route (more nets, more crowding)
Fab/assembly cost	More boards = more per-unit fab/panel costs, though a single small design panelizes cheaply; more connectors and enclosures needed	Fewer, larger boards; fewer connectors overall, which also means fewer failure points at connections
Space/mechanical integration	Needs enclosure/mounting at each joint — real mechanical design burden on a humanoid arm where joint housings are already tight	Central board fits in the torso/upper-arm where there's comparatively more room; joints stay mechanically simpler
EMI/noise	Motor driver switching noise stays localized near its own motor, less coupling between axes	All drivers switching on one board — more traces to manage for ground-loop/noise isolation, though your plan already accounts for this (Section 4.5's star-grounding note)
Firmware/bus architecture	Every joint is a CAN (or similar) node — clean, modular, but more nodes to address/manage, more bus traffic	One MCU owns several joints directly over local SPI/UART — simpler bus topology, less network overhead
Cost at your scale (10-12 DOF)	Meaningfully more expensive: 10-12 small PCBs, each needing its own regulator/connectors/enclosure vs. sharing those on one board	Cheaper — shared regulation, shared connectors, less redundant circuitry
Time (your 1-semester constraint)	Slower to bring up — more individual boards to fab, assemble, test, and debug in parallel	Matches your stated plan and timeline much better — fewer boards to design/fab/debug in a tight schedule

12.3 Where each one makes sense

Per-motor PCBs are the right call when: joints are very physically separated (e.g., a full-body humanoid with legs, long cable runs), when you want hot-swappable joint modules for a product-like design, or when a research team is optimizing for fault isolation and modularity over cost/time (this is closer to how Dynamixel-style "smart actuators" work internally, conceptually).

Consolidated boards are the right call when: DOF count is moderate (your 10-12 joints across two arms), the mechanical structure is compact enough that long central wiring runs are manageable, and — critically for you — **time and cost are constrained by a one-semester deadline and a mid-size budget.**

12.4 Recommendation

Given your actual constraints, I'd keep the plan as currently written (Option B, consolidated per-subsystem boards), not move to per-motor dedicated PCBs. The wiring/fault-isolation benefits of per-motor boards are real, but they cost you meaningfully more design, fab, and integration time — the resource you have the least slack on. If a specific joint proves to be a persistent problem during bring-up (e.g., one arm's board damaged, blocking the whole arm), that's the point where a targeted per-joint board for just that axis becomes worth the extra effort — not as a default architecture from day one.